

Khant Thu Aung / 105292912

5.2P

Bag.cs

```
namespace SwinAdventure
```

```
{
```

```
    public class Bag : Item
```

```
    {
```

```
        private Inventory _inventory;
```

```
        public Bag(string[] ids, string name, string desc) : base(ids, name, desc)
```

```
        {
```

```
            _inventory = new Inventory();
```

```
        }
```

```
        public GameObject Locate(string id)
```

```
        {
```

```
            if (AreYou(id))
```

```
            {
```

```
                return this;
```

```
            }
```

```
            return _inventory.Fetch(id);
```

```
        }
```

```
        public override string FullDescription
```

```
        {
```

```
            get
```

```
            {
```

```

        string itemsDescription = _inventory.ItemList;
        if (string.IsNullOrEmpty(itemsDescription))
        {
            return $"{base.FullDescription}. It's empty.";
        }
        else
        {
            return $"{base.FullDescription}. In the {Name}, you can
see:\n{itemsDescription}";
        }
    }
}

```

```

public Inventory Inventory
{
    get { return _inventory; }
}
}

```

TestBag.cs

```

using SwinAdventure;
using NUnit.Framework.Legacy;
namespace UnitTest
{
    public class TestBag
    {

```

[Test]

```
public void TestBagLocatesItems()
```

```
{
```

```
    Bag bag = new Bag(new string[] { "bag", "smallbag" }, "Small Bag", "A small bag");
```

```
    Item item1 = new Item(new string[] { "sword", "a short sword" }, "Sword", "A beautiful  
short sword.");
```

```
    bag.Inventory.Put(item1);
```

```
    GameObject foundItem = bag.Locate("sword");
```

```
    ClassicAssert.AreEqual(item1, foundItem);
```

```
}
```

[Test]

```
public void TestBagLocatesItself()
```

```
{
```

```
    Bag bag = new Bag(new string[] { "bag", "smallbag" }, "Small Bag", "A small bag");
```

```
    GameObject foundBag = bag.Locate("bag");
```

```
    ClassicAssert.AreEqual(bag, foundBag);
```

```
}
```

[Test]

```
public void TestBagLocatesNothing()
```

```
{
```

```
    Bag bag = new Bag(new string[] { "bag", "smallbag" }, "Small Bag", "A small bag");
```

```
    GameObject foundNothing = bag.Locate("bigbag");
```

```
    ClassicAssert.IsNull(foundNothing);
```

```
}
```

[Test]

```
public void TestBagFullDescription()
{
    Bag bag = new Bag(new string[] { "bag", "smallbag" }, "Small Bag", "A small bag");

    Item item1 = new Item(new string[] { "sword", "a short sword" }, "Sword", "A beautiful
short sword.");

    bag.Inventory.Put(item1);

    string description = bag.FullDescription;

    ClassicAssert.IsTrue(description.Contains("A small bag."));

    ClassicAssert.IsTrue(description.Contains("In the Small Bag, you can see:\n"));

    ClassicAssert.IsTrue(description.Contains("Sword (sword)"));

}
```

[Test]

```
public void TestBagInBag()
{
    Bag b1 = new Bag(new string[] { "bag1", "bigbag" }, "Big Bag", "A big bag");

    Bag b2 = new Bag(new string[] { "bag2", "smallbag" }, "Small Bag", "A small bag");

    Item sword = new Item(new string[] { "sword", "short sword" }, "Sword", "A beautiful
short sword.");

    b1.Inventory.Put(b2);

    b1.Inventory.Put(sword);

    var foundBag2InBag1 = b1.Locate("smallbag");

    var foundItem1InBag1 = b1.Locate("sword");
```

```

var foundItem1InBag2 = b2.Locate("sword"); // Should return null

ClassicAssert.AreEqual(b2, foundBag2InBag1);

ClassicAssert.AreEqual(sword, foundItem1InBag1);

ClassicAssert.IsNull(foundItem1InBag2);
}

[Test]
public void TestBagInBagWithPrivilegedItem()
{
    Bag b1 = new Bag(new string[] { "bag1", "bigbag" }, "Big Bag", "A big bag");
    Bag b2 = new Bag(new string[] { "bag2", "smallbag" }, "Small Bag", "A small bag");
    Item privilegedItem = new Item(new string[] { "sword", "short sword" }, "Sword", "A
beautiful short sword.");

    b2.Inventory.Put(privilegedItem);
    b2.PrivilegeEscalation("2912");
    b1.Inventory.Put(b2);

    GameObject foundPrivilegedItemInBag1 = b1.Locate("sword");
    ClassicAssert.IsNull(foundPrivilegedItemInBag1);
}
}
}

```

